



Distribution-aware sampling of replay buffer for mitigating catastrophic forgetting

Armaan Garg^{a,*}, Ansh Raj Sharma^b, Aryan Arora^b, Shilpa Verma^{b,*}

^a Department of Computer Science and Engineering, Dr. B. R. Ambedkar, National Institute of Technology (NIT), Jalandhar, 144008, India

^b Department of Computer Science and Engineering, Punjab Engineering College, Chandigarh, 160012, India

ARTICLE INFO

Keywords:

Reinforcement learning
Catastrophic forgetting
Experience replay
K-means
Gymnasium

ABSTRACT

Experience replay allows reinforcement learning agents to maintain a buffer of past transitions, from which samples are repeatedly drawn during training. This reduces temporal correlations and improves learning stability and sample efficiency. Conventional sampling methods typically draw transitions from the replay buffer either uniformly or based on priority measures, without explicitly considering the distribution of the state-action space. This can result in imbalanced sampling and increase the risk of catastrophic forgetting, where previously learned behaviors are overwritten by new information. We propose four sampling strategies to identify a generalizable solution, where the K-Means based state-action space aware approach outperforms others. The method partitions the transitions (s, a) into $\{C_k\}_{k=1}^K$ using K-Means, and samples are drawn from each cluster to ensure distribution-aware and representative sampling of the state-action space. We evaluate established benchmark methods, including DQN and PER, along with our proposed variants across various Gymnasium environments, including Classic Control and Atari. The results demonstrate the effectiveness of the proposed approaches in mitigating catastrophic forgetting and enhancing learning stability.

1. Introduction

Deep reinforcement learning (DRL) [1] has shown remarkable success in solving complex sequential decision-making tasks [2,3] by enabling agents to learn directly from raw experiences. However, unlike supervised learning, where data samples are typically assumed to be independent and identically distributed (i.i.d.), reinforcement learning agents collect experiences in a temporally correlated manner as they interact with the environment. This sequential correlation violates the i.i.d. assumption required by standard gradient-based optimization methods. Furthermore, the continual change in learning targets introduces instability in value function approximation, often causing the agent to diverge or exhibit unstable learning behavior.

To address these challenges, experience replay buffers [4,5] and the use of dual networks have been widely adopted [6,7]. Building on Q-learning [8], an off-policy algorithm, the Deep Q-Network (DQN) framework [1] introduced several key innovations to enable stable and scalable learning with function approximators. First, storing past experiences in a replay buffer allows the agent to sample them randomly, thereby breaking the correlation between consecutive samples and supporting more stable gradient-based updates. Second, maintaining two separate networks—a policy network that is updated frequently and a target network that is updated periodically—provides a more stable learning target, reducing approximation-induced instability. These enhancements formed the foundation of the DQN algorithm, which has demonstrated the ability to learn and play a wide range of classic control and Atari

* Corresponding authors.

Email addresses: garga@nitj.ac.in (A. Garg), shilpa@pec.edu.in (S. Verma).

<https://doi.org/10.1016/j.ins.2026.123640>

Received 25 November 2025; Received in revised form 13 May 2026; Accepted 13 May 2026

Available online 16 May 2026

0020-0255/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

games [9]. Despite its success, DQN suffers from the issue of *catastrophic forgetting* [10,11], where the model gradually loses previously acquired knowledge while learning from new sequential experiences. This limitation primarily stems from the inconsistent update of network parameters and the finite capacity of the experience replay buffer, leading to overwriting of old information and a loss of long-term memory.

The rest of the paper is organized as follows. Section 2 reviews related work on strategies for optimizing experience replay buffers in reinforcement learning. Section 3 provides background on DQN and discusses the phenomenon of *catastrophic forgetting*. Section 4 presents the methodology and architectural details of the four proposed techniques: average reward thresholding, average priority thresholding, a combined reward-priority approach, and k-means experience replay. Section 5 details the experimental settings, describes the evaluation environments, and presents the results. Finally, Section 6 concludes the paper.

2. Related literature

One of the fundamental components enabling the success of deep reinforcement learning (DRL) algorithms is the experience replay buffer mechanism, initially proposed in Deep Q-Networks (DQN) [1], which extends standard Q-learning methods. By storing past transitions, represented as state, action, reward, and next state tuples in a replay buffer and sampling small batches during training, DRL agents can efficiently reuse past experiences. This approach improves data efficiency, stabilizes learning, and accelerates convergence towards optimal policies.

Conventionally, DQN (Deep Q-Networks) samples experience tuples $\langle s_t^{(i)}, a_t^{(i)}, r_{t+1}^{(i)}, s_{t+1}^{(i)} \rangle$ uniformly at random from the replay buffer during training. However, uniform sampling assumes all experiences contribute equally, which often leads to redundant updates. Many transitions, especially those involving states with low temporal-difference (TD) errors, are repeatedly sampled, while rare but informative transitions with high rewards or large TD errors are not sampled frequently. To overcome the limitations of uniform sampling and promote faster convergence, more advanced replay buffer architectures have been introduced.

One prominent advancement is Prioritized Experience Replay (PER) [12], which addresses experience redundancy by prioritizing transitions with higher TD errors and sampling them more frequently. This ensures that critical experiences are more likely to influence the learning process. To avoid greedily selecting only a narrow subset of high-priority transitions and to mitigate the bias introduced by non-uniform sampling, PER incorporates stochastic prioritization. Rather than deterministically selecting transitions with the largest TD errors, it assigns sampling probabilities proportional to the TD-error-based priority, ensuring that even low-priority transitions maintain a non-zero chance of being replayed.

Despite its effectiveness, PER exhibits certain limitations. For instance, transitions near the action space boundaries often display large TD errors, yet contribute little to accelerating the learning process [13]. This exposes a key weakness of relying solely on current TD errors, as they do not always reflect the long-term utility of a transition. Consequently, research in this area diverges into two primary directions: improving the selection of which transitions to store in the replay buffer and refining the methods used to sample the most relevant experiences.

Recent works explore alternative scoring criteria beyond TD error to prioritize stored transitions more effectively. For example, BETDQNet introduces a method that combines both TD error and Bellman Error (BE) within an adaptive weighted scoring function, enabling simultaneous consideration of both criteria for experience prioritization [14]. Similarly, RPE-PER [15] prioritizes experiences based on Reward Prediction Error—the difference between predicted and actual rewards—leveraging an auxiliary critic network (EMCN) alongside standard critic networks to improve reward predictions. Another notable approach, proposed in Prioritizing Samples in Reinforcement Learning with Reducible Loss [16], introduces a learnability-driven metric called reducible loss (ReLo). This method prioritizes experiences that exhibit steady reductions in training loss over time, as these transitions signal meaningful learning opportunities.

Despite extensive progress in improving replay efficiency [17,18], existing clustering based sampling strategies remain constrained by localized or heuristic partitioning of experiences. Replay memory design continues to play a crucial role in determining the stability and data efficiency of deep reinforcement learning, yet prevailing methods such as Clustering Experience Replay (CER) [19] and K-Means Replay Memory (KMR) [20] largely focus on temporal or loss-based clustering. CER divides training into temporal segments and applies K-Means clustering within each period to enhance short-term exploration–exploitation balance, while KMR organizes experiences into hierarchical memory buffers and samples according to loss-weighted cluster probabilities. Although these approaches improve sampling diversity, they overlook the global topology of the state-action distribution, leading to uneven state coverage and persistent forgetting of underrepresented transitions. To address this limitation, our proposed K-Means Driven Sampling for Experience Replay (K-MER) introduces a state-action space aware framework that directly clusters transitions across the entire state-action manifold. By enforcing distribution-aware and representative sampling, K-MER preserves replay diversity, mitigates *catastrophic forgetting*, and stabilizes long-horizon learning without relying on temporal segmentation or complex probabilistic weighting.

3. Catastrophic forgetting in DQN from random sampling

In this section, we provide a brief overview of the DQN model and examine the phenomenon of catastrophic forgetting caused by uniform random sampling from the experience replay buffer.

Deep Q Networks (DQN) [1] is a model-free off-policy learning algorithm that uses a pair of neural networks: a *learning network* and a *target network*. Both networks are initialized with the same weights but are updated at different frequencies. Each network takes the agent's state as input and approximates the Q-value function over feasible actions. DQN follows an ϵ -greedy policy, where initially the agent explores the environment via random actions. Over time, it stores transitions $\langle s_t, a_t, r_t, s_{t+1} \rangle$ in a replay buffer $\mathcal{D}$. Random mini-batches are drawn from this buffer to train the learning network.

The TD loss gradient at iteration t is:

$$\nabla_{\theta_t} \mathcal{L}_t(\theta_t) = \mathbb{E}_{s,a \sim \rho, s' \sim p} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_{t-1}) - Q(s, a; \theta_t) \right) \nabla_{\theta_t} Q(s, a; \theta_t) \right] \quad (1)$$

where $\rho(i)$ denotes the probability of sampling transition i from buffer $\mathcal{D}$.

Note: In practice, the buffer fills gradually and old transitions are replaced once full. For simplicity, we assume the buffer has reached capacity and all N transitions remain available throughout the K updates (i.e., no replacement occurs).

Lemma 1 (Under-Sampling of Informative Transitions). *Let $\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N$ be a replay buffer, and let transitions be sampled i.i.d. with arbitrary probabilities $\rho(i)$, where $\sum_{i=1}^N \rho(i) = 1$. Then the expected number of updates to a transition $(s_j, a_j, r_j, s'_j) \in \mathcal{D}$ over K training steps is:*

$$\mathbb{E}[\# \text{ updates to } j] = K \cdot \rho(j).$$

If $\rho(j)$ is small (e.g., $\rho(j) = \frac{1}{N}$ under uniform sampling), then the expected number of updates received by transition j may remain very low when N is large relative to K , causing informative transitions to become underrepresented during learning.

Proof. Let $X_k^{(j)}$ be an indicator variable:

$$X_k^{(j)} = \begin{cases} 1, & \text{if transition } j \text{ is sampled at step } k, \\ 0, & \text{otherwise.} \end{cases}$$

Then the total number of times transition j is sampled is:

$$U_j = \sum_{k=1}^K X_k^{(j)}.$$

Taking expectation:

$$\mathbb{E}[U_j] = \sum_{k=1}^K \mathbb{E}[X_k^{(j)}] = \sum_{k=1}^K \rho(j) = K \cdot \rho(j).$$

Therefore, if $\rho(j) \ll 1$, then $\mathbb{E}[U_j] \ll K$, and j receives few updates. In particular, under uniform sampling $\rho(j) = \frac{1}{N}$, and $\mathbb{E}[U_j] = \frac{K}{N}$. Thus, when N is large, each transition receives relatively few updates, which can lead to forgetting. $\square$

Furthermore, in standard DQN, as older experiences are gradually replaced by newer ones in the replay buffer, the likelihood of forgetting increases even more, since the agent loses access to important past transitions and their associated state distributions weaken over time.

4. Proposed approaches

In this section, we discuss our novel approaches—namely, average reward thresholding, average priority thresholding, a combination of reward and priority, and k-means experience replay—to address *catastrophic forgetting*. Our objective is to reduce catastrophic forgetting in DQN while minimizing memory usage [21]. The experience replay buffer $\mathcal{D}$ is maintained with a fixed capacity N , such that only the most relevant transitions are retained for training. The proposed approaches aim to make the sampling process more state-aware by prioritizing transitions based on their estimated utility, thereby improving learning stability and efficiency even under limited buffer constraints.

4.1. DQN with average reward thresholding

The average reward thresholding technique addresses the imbalance in experience replay by partitioning the replay buffer into two hierarchical levels, denoted as $\mathcal{H}_1$ (high priority) and $\mathcal{H}_2$ (low priority). A dynamic threshold $\mathcal{R}$ is computed at each time step to separate transitions with high and low rewards. This threshold is calculated as the mean of all rewards present in both buffer levels, formally defined as:

$$\mathcal{R} = \frac{1}{N} \sum_{i=1}^N r_i \quad (2)$$

where N is the total number of transitions stored in $\mathcal{H}_1$ and $\mathcal{H}_2$, and r_i is the reward associated with the i -th transition.

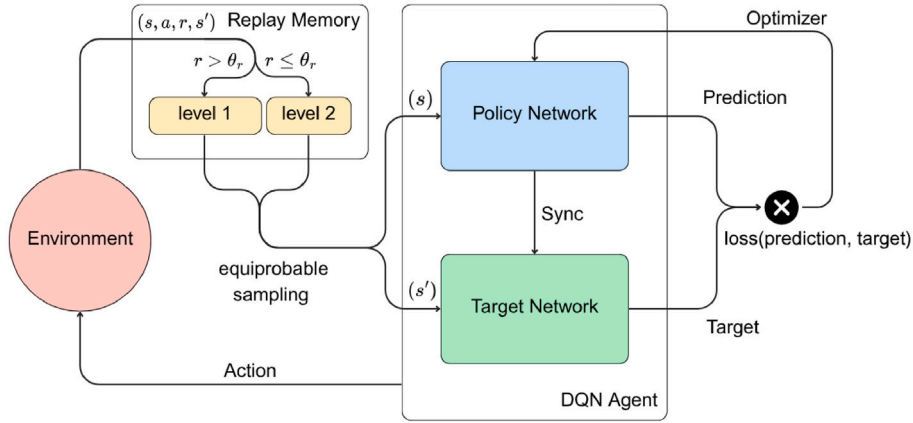
When a new experience arrives, it is compared to $\mathcal{R}$. If its reward exceeds $\mathcal{R}$, it is inserted into $\mathcal{H}_1$; otherwise, it is placed in $\mathcal{H}_2$. Sampling is then performed uniformly across both buffers, proportionally to the batch size. This promotes a balanced exploration of both high- and low-reward experiences, while maintaining sufficient diversity for robust policy updates. The model continues training on these buffered samples throughout the duration of an episode. The workflow of this technique is summarized in [Algorithm 1](#) and an illustrative overview of the architecture is given in [Fig. 1](#). This proposed approach is able to mitigate catastrophic forgetting to some extent as compared to standard DQN (for details refer to [Section 5.2](#)).

Algorithm 1 DQN with Average Reward Threshold.

```

1: Input: Total buffer size  $N$ , level ratio  $l$ , batch size  $B$ , discount factor  $\gamma$ , learning rate  $\eta$ ,  $\mathcal{H}_1$  (size  $lN$ ),  $\mathcal{H}_2$  (size  $(1-l)N$ ); networks  $Q_\theta, Q_{\theta^-}$ ; threshold  $\tau \leftarrow 0$ 
2: for each episode do
3:   Reset environment; observe  $s_0$ 
4:   for each step  $t$  do
5:     Select action  $a_t \sim \epsilon\text{-greedy}(Q_\theta(s_t))$ ; observe  $r_t, s_{t+1}, f_t$ 
6:     Update threshold:  $\tau \leftarrow \frac{1}{N} \sum_{i=1}^N r_i$ 
7:     Store  $(s_t, a_t, r_t, s_{t+1}, f_t)$  in  $\mathcal{H}_1$  if  $r_t > \tau$ , else in  $\mathcal{H}_2$ 
8:     if sufficient samples available then
9:       Sample  $\mathcal{B} \propto \mathcal{H}_1, \mathcal{H}_2$ 
10:      for each  $(s, a, r, s', f) \in \mathcal{B}$  do
11:         $y \leftarrow r + \gamma(1-f) \max_{a'} Q_{\theta^-}(s', a')$ 
12:         $\delta \leftarrow y - Q_\theta(s, a)$ 
13:       $\mathcal{L}(\theta) \leftarrow \begin{cases} \frac{1}{2} \delta^2, & |\delta| < 1 \\ |\delta| - \frac{1}{2}, & \text{otherwise} \end{cases}$ 
14:      end for
15:      Update parameters:  $\theta \leftarrow \theta + \eta \nabla_\theta \mathcal{L}(\theta)$ 
16:    end if
17:    Periodically update target:  $\theta^- \leftarrow \theta$ 
18:  end for

```

**Fig. 1.** Average reward threshold architecture.**4.2. DQN with average priority thresholding**

As with the average reward technique, the average priority thresholding also consists of two buffer levels $\mathcal{H}_1$ (high) and $\mathcal{H}_2$ (low). In this a priority $\rho(s)$ is assigned to each sample, defined as:

$$\rho(s) = \left(\min_{s_j \in S} \|s - s_j\|_2 \right)^\alpha \quad (3)$$

where S is the set of already stored states, $\|\cdot\|_2$ denotes the Euclidean norm, and α controls the scaling of the priority.

A dynamic threshold is calculated using buffer priorities. The threshold is the average of all the priorities present in the two buffers, defined as:

$$\mathcal{T} = \frac{1}{N} \sum_{i=1}^N \rho_i \quad (4)$$

with N representing the total number of transitions stored. When a new sample arrives, if its priority exceeds $\mathcal{T}$, it is inserted into $\mathcal{H}_1$; otherwise, it is stored in $\mathcal{H}_2$. Probabilistic sampling is applied to train the model, where priorities are used as weights to sample a batch of transitions. The overall flow of this technique is summarized in [Algorithm 2](#), and the architecture of the proposed approach is provided in [Fig. 2](#). However, this approach did not demonstrate the same level of effectiveness as the average reward thresholding technique when evaluated on the MountainCar-v0 environment (for details refer to [Section 5.3](#)).

Algorithm 2 DQN with Average Priority Thresholding.

```

1: Input: Priority exponent  $\alpha$ ; priority lists  $\mathcal{P}_1, \mathcal{P}_2$ 
2: for each episode do
3:   Reset environment; observe  $s_0$ 
4:   for each step  $t$  do
5:     Select  $a_t \sim \epsilon$ -greedy( $Q_\theta(s_t)$ ); observe  $r_t, s_{t+1}, f_t$ 
6:     Compute priority:  $\rho_t = \left( \min_{s_i \in \mathcal{S}} \|s_t - s_i\|_2 \right)^\alpha$ 
7:     Compute threshold:  $\tau \leftarrow \frac{1}{|\mathcal{P}_1| + |\mathcal{P}_2|} (\sum \mathcal{P}_1 + \sum \mathcal{P}_2)$ 
8:     Store transition in:
       
$$\begin{cases} \mathcal{H}_1, \mathcal{P}_1 \leftarrow \mathcal{P}_1 \cup \{\rho_t\}, & \rho_t > \tau \\ \mathcal{H}_2, \mathcal{P}_2 \leftarrow \mathcal{P}_2 \cup \{\rho_t\}, & \text{otherwise} \end{cases}$$

9:   if sufficient samples available then
10:     $Z \leftarrow \sum \mathcal{P}_1 + \sum \mathcal{P}_2$ 
11:     $B_1 \leftarrow \lfloor B \cdot \frac{\sum \mathcal{P}_1}{Z} \rfloor, B_2 \leftarrow B - B_1$ 
12:    Sample  $B_1$  from  $\mathcal{H}_1, B_2$  from  $\mathcal{H}_2$  using respective priority weights
13:    for each  $(s, a, r, s', f) \in \mathcal{B}$  do
14:       $y \leftarrow r + \gamma(1-f) \max_{a'} Q_{\theta^-}(s', a')$ 
15:       $\delta \leftarrow y - Q_\theta(s, a)$ 
16:       $\mathcal{L}(\theta) \leftarrow \begin{cases} \frac{1}{2} \delta^2, & |\delta| < 1 \\ |\delta| - \frac{1}{2}, & \text{otherwise} \end{cases}$ 
17:    end for
18:    Update  $\theta \leftarrow \theta + \eta \nabla_\theta \mathcal{L}(\theta)$ 
19:  end if
20:  Periodically update target:  $\theta^- \leftarrow \theta$ 
21: end for
22: end for

```

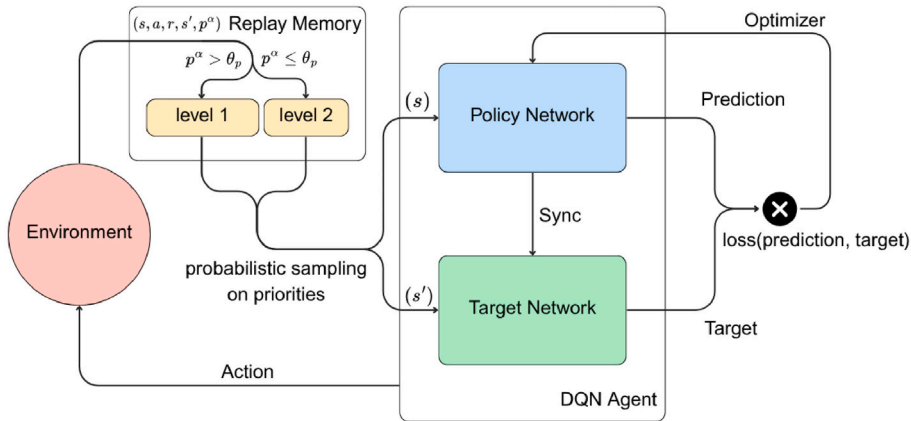


Fig. 2. Average priority threshold architecture.

4.3. DQN with combined reward-priority thresholding

This approach integrates the average reward thresholding and average priority thresholding methods. The dynamic reward threshold is computed as the average of all rewards present in the two buffers, as defined in Eq. (2). Each incoming transition is assigned a priority according to Eq. (3). If the reward of a new sample exceeds the threshold, it is inserted into $\mathcal{H}_1$; otherwise, it is stored in $\mathcal{H}_2$. For sampling, the priorities are used as probabilistic weights to select a batch of transitions to train the model. This combined strategy aims to balance high-reward experiences with state novelty, improving the efficiency and stability of policy learning. The detailed procedure is summarized in Algorithm 3 and visual representation of the model's architecture is given in Fig. 3. This combined technique performed slightly better than the standalone average priority thresholding approach, yet remained less effective compared to the average reward thresholding method (for details refer to Section 5.4).

Algorithm 3 DQN with Combined Reward-Priority Thresholding.

```

1: Input: Priority exponent  $\alpha$ ; priority lists  $\mathcal{P}_1, \mathcal{P}_2$ ; reward list  $\mathcal{R}$ 
2: for each episode do
3:   Reset environment; observe  $s_0$ 
4:   for each step  $t$  do
5:     Select  $a_t \sim \epsilon$ -greedy( $Q_\theta(s_t)$ ); observe  $r_t, s_{t+1}, f_t$ 
6:     Append  $r_t$  to  $\mathcal{R}$ ; compute reward threshold  $\tau_r = \frac{1}{|\mathcal{R}|} \sum \mathcal{R}$ 
7:     Set level:  $\text{level} \leftarrow \mathbb{I}[r_t \leq \tau_r]$ 
8:     Compute priority:  $\rho_t = \left( \min_{s_i \in \mathcal{S}} \|s_t - s_i\|_2 \right)^\alpha$ 
9:     Store transition in  $\mathcal{H}_{\text{level}}$ ; append  $\rho_t$  to  $\mathcal{P}_{\text{level}}$ 
10:    if buffers ready then
11:       $Z \leftarrow \sum \mathcal{P}_1 + \sum \mathcal{P}_2$ 
12:       $B_1 \leftarrow \lfloor B \cdot \frac{\sum \mathcal{P}_1}{Z} \rfloor, B_2 \leftarrow B - B_1$ 
13:      Sample from  $\mathcal{H}_1, \mathcal{H}_2$  using weights  $\mathcal{P}_1, \mathcal{P}_2$ 
14:      for each  $(s, a, r, s', f) \in \mathcal{B}$  do
15:         $y \leftarrow r + \gamma(1-f) \max_{a'} Q_{\theta^-}(s', a')$ 
16:         $\delta \leftarrow y - Q_\theta(s, a)$ 
17:         $\mathcal{L}(\theta) \leftarrow \begin{cases} \frac{1}{2} \delta^2, & |\delta| < 1 \\ |\delta| - \frac{1}{2}, & \text{otherwise} \end{cases}$ 
18:      end for
19:      Update  $\theta \leftarrow \theta + \eta \nabla_\theta \mathcal{L}(\theta)$ 
20:    end if
21:    Periodically update target:  $\theta^- \leftarrow \theta$ 
22:  end for
23: end for

```

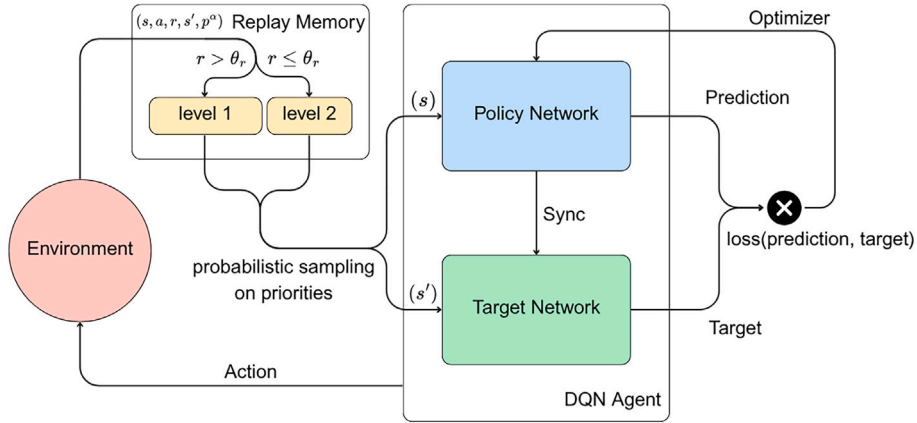


Fig. 3. Combined Average Reward-Priority Threshold Architecture.

4.4. Deep Q-learning with K-means experience replay

The K-Means state-action space aware sampling technique (K-MER) leverages clustering to restructure and organize the replay buffer before extracting transition samples. Specifically, the state-action space is partitioned into B clusters using the K-Means algorithm, where B denotes the batch size. By grouping similar state-action pairs together, the K-Means process captures the underlying structure of the environment and ensures adequate coverage across diverse regions of the state-action space.

During the initial exploration phase, the agent samples transitions uniformly from each cluster:

$$(s, a, r, s', f) \sim \text{Uniform}(C_k), \quad \forall k = 1, \dots, B$$

This guarantees that all areas of the state-action space, including sparsely visited or low-frequency regions are adequately explored.

As training advances, the agent transitions to an exploitation phase by selecting the highest-reward experience within each cluster:

$$(s, a, r, s', f) = \arg \max_{i \in C_k} r_i$$

Algorithm 4 Deep Q-Learning with K-Means Experience Replay (K-MER).

```

1: Input: Initial exploration rate  $\epsilon$ , decay  $\epsilon_{\text{decay}}$ ,  $\min \epsilon_{\min}$ , buffer  $D$ , sync interval  $T_{\text{sync}}$ 
2: for each episode do
3:   Reset environment; observe  $s_0$ 
4:   for each step  $t$  do
5:     Select  $a_t \sim \epsilon\text{-greedy}(Q_\theta(s_t))$ ; observe  $r_t, s_{t+1}, f_t$ 
6:     Store  $(s_t, a_t, r_t, s_{t+1}, f_t)$  in  $D$ 
7:     if  $|D| \geq B$  then
8:       if  $t \bmod T_{\text{sync}} = 0$  then
9:         Cluster transitions  $(s, a)$  into  $\{C_k\}_{k=1}^K$  using K-Means
10:        for each cluster  $C_k$  do
11:          Select:

$$(s_k, a_k, r_k, s'_k, f_k) \sim \begin{cases} \text{random sample} \in C_k, & \text{w.p. } \epsilon \\ \arg \max_i r_i \in C_k, & \text{w.p. } 1 - \epsilon \end{cases}$$

12:        end for
13:        Aggregate selected samples into batch  $B$ 
14:      else
15:        Sample random batch  $B$  from  $D$ 
16:      end if
17:      for each  $(s_j, a_j, r_j, s'_j, f_j) \in B$  do
18:         $y_j \leftarrow r_j + \gamma(1-f_j) \max_{a'} Q_{\theta^-}(s'_j, a')$ 
19:         $\delta_j \leftarrow y_j - Q_\theta(s_j, a_j)$ 
20:         $\mathcal{L}_j(\theta) \leftarrow \begin{cases} \frac{1}{2} \delta_j^2, & |\delta_j| < 1 \\ |\delta_j| - \frac{1}{2}, & \text{otherwise} \end{cases}$ 
21:      end for
22:      Update  $\theta \leftarrow \theta + \eta \nabla_\theta \mathcal{L}(\theta)$ 
23:    end if
24:    if  $t \bmod T_{\text{sync}} = 0$  then
25:      Update target:  $\theta^- \leftarrow \theta$ 
26:    end if
27:  end for
28:  Decay  $\epsilon \leftarrow \max(\epsilon_{\min}, \epsilon \cdot \epsilon_{\text{decay}})$ 
29: end for

```

This cluster-based sampling framework enables a smooth shift from exploration to exploitation, promoting both robust state-action coverage and efficient learning. The detailed approach is outlined in Algorithm 4 and the proposed architecture is illustrated in Fig. 4. The code files can be found in the given GitHub repository ¹.

Computational Overhead: K-Means clustering is performed only when the target network is updated, rather than at every gradient step, limiting its frequency and avoiding a per-step computational cost. Moreover, K-Means is not applied to raw pixel observations. For Atari and CarRacing-v3 (Box2D) environments, clustering is performed on the flattened output of the CNN encoder instead of the original pixel space. With a fixed number of clusters K and a bounded replay subset, the computational complexity of K-MER scales as $\mathcal{O}(K \cdot d \cdot n)$ per synchronization step, where d denotes the embedding dimension and n is the number of sampled transitions. Under these conditions, the additional clustering overhead remains small relative to the overall training cost.

5. Results

We evaluate standard DQN and the proposed approaches on various environments including Classic, Box2D and Atari. A total of 8 environments are selected, the summary of which is provided in Table 1. The standard DQN is evaluated on all the environments to verify *catastrophic forgetting*, however our proposed technique is tested on the environments with most noticeable forgetting, namely, MountainCar-v0, LunarLander-v2 and CarRacing-v3. All the evaluations are conducted for a total of 5000 episodes. The aggregate reward in an episode versus the number of episodes is plotted for each environment and for different approaches. Further, Table 2 provides the hyperparameters and their selected value ranges used for training the models.

For the proposed model, K-MER, we choose K (number of clusters) = B (mini-batch size) so that clustering acts as a structured replacement for uniform mini-batch sampling: each gradient step still uses exactly B transitions, but selection is guided by state-space structure. Selecting one representative transition from each cluster ensures that the resulting batch contains samples drawn from

¹ <https://github.com/anonpersonal/InformationSciences.git>.

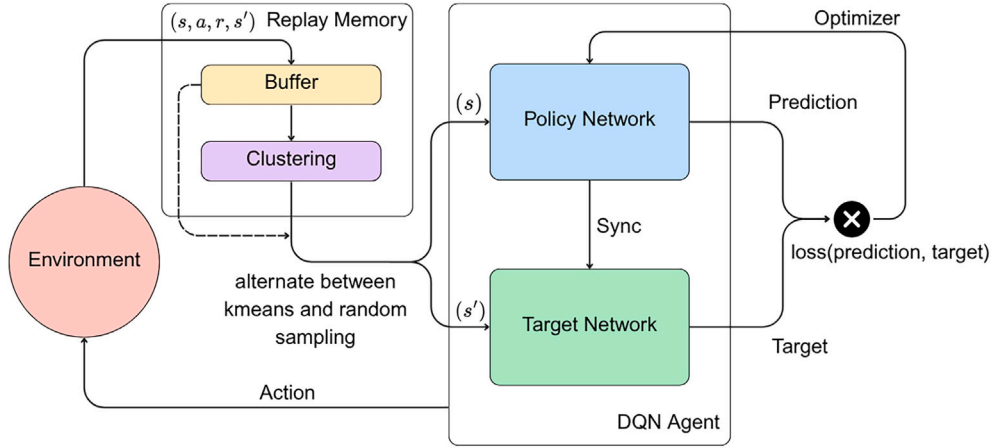


Fig. 4. DQN with K-means experience replay architecture (K-MER).

Table 1
Summary of different environments evaluated in our work.

Type	Environment	Description
Classic	MountainCar-v0	Reach the top of the right hill (goal).
	CartPole-v1	Keep the pole upright on the cart.
	Acrobot-v1	Reach the target height in minimum steps.
Box2D	LunarLander-v2	Land the lander safely on the ground.
	CarRacing-v3	Drive the car along the track using pixel observations.
Atari	Breakout-v5	Destroy maximum brick walls.
	MsPacman-v5	Collect all pellets and avoid ghosts.
	Pong-v5	Score goals while blocking opponent's.
	SpaceInvaders-v5	Shoot opponents max times in games.

Table 2
Hyperparameter configurations used in the experiments.

Hyperparameter	Range of Values	Description
γ	0.95 - 0.99	Discount Factor
α	0.5, 0.6, 0.7, 0.8	Priority Scale
B	32, 64, 128	Batch Size
η	0.0001	Learning Rate
N	10,000	Standard Memory Size
H_1	3000 - 5000	High Memory Size
H_2	5000 - 7000	Low Memory Size
l	0.3, 0.4, 0.5	Level Ratio

different regions of the stored state space. This preserves compatibility with standard DQN training while avoiding the introduction of an additional batch-size hyperparameter.

5.1. Standard DQN evaluation

The classic implementation of DQN is tested across all the environments to check the level of *catastrophic forgetting* each environment exhibits. Fig. 5 indicates that simple classic environments, such as, Acrobot-v1 and CartPole-v1 don't show significant *catastrophic forgetting*. Another classic environment, MountainCar-v0, demonstrates significant *catastrophic forgetting*. The MountainCar-v0 environment suffers immensely as it observes prominent dips around 2000 episodes and after 3000. The DQN model seems to completely forget what it had previously learned. In Box2D environment, LunarLander-v2, the classic DQN suffers performance breakdowns twice, around 1500 and 2500 episodes, but nothing as significant compared to MountainCar-v0.

On the other hand, the Atari environments, namely, Breakout-v5, Ms-Pacman-v5, Pong-v5, SpaceInvaders-v5, also display minimal *catastrophic forgetting* at different stages. Although, for such complex environments the number of episodes they are evaluated on is limited. This may be one of the reasons why significant forgetting is not being observed in these environments. Out of these four environments, Breakout-v5 forgets around 3500 episodes, Ms-Pacman-v5 forgets twice around 3000 and 4000 episodes, while Pong-v5 and SpaceInvaders-v5 showed quite a linear trend but dipped around 4000 episodes.

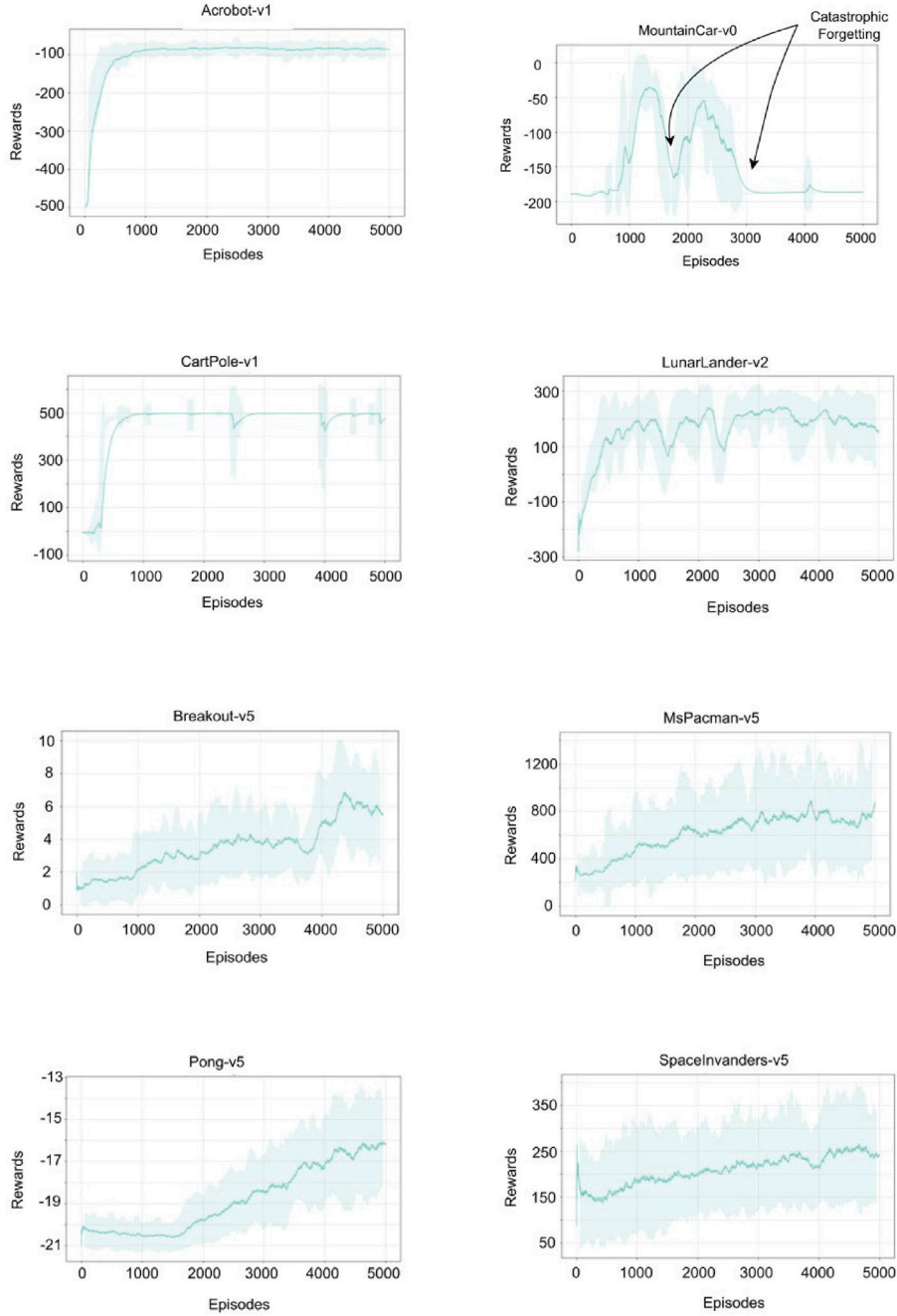


Fig. 5. Implementation of standard DQN on Gymnasium environments.

Based on these results, we focus our subsequent experiments on MountainCar-v0, which showed the most severe catastrophic forgetting. To quantify catastrophic forgetting, we define a *forgetting event* based on sustained performance degradation. Let Y denote the running average reward computed over a sliding window of 100 episodes. After the agent first exceeds the performance threshold -110 (representing a standard success threshold in MountainCar-v0 environment), a forgetting event is recorded whenever $Y < -110$ for at least 50 consecutive episodes. The total number of such events during training is reported as the *forgetting count* ($\mathbb{F}$). This filters out short-term fluctuations and captures only persistent performance collapse, providing a quantitative measure of long-horizon policy retention.

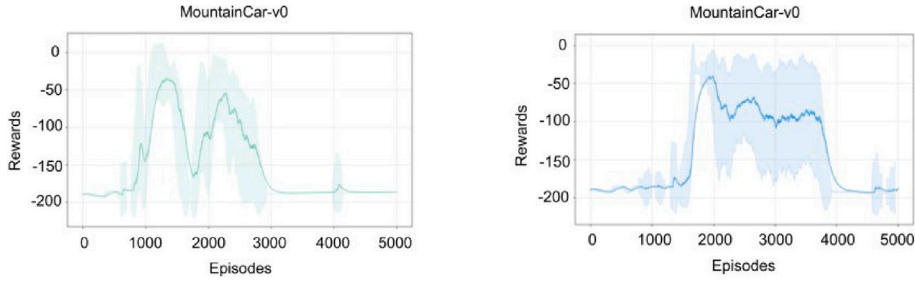


Fig. 6. Comparison of standard DQN and average reward thresholding DQN on the MountainCar-v0 environment. Left (in green): standard DQN. Right (in blue): DQN with average reward thresholding.

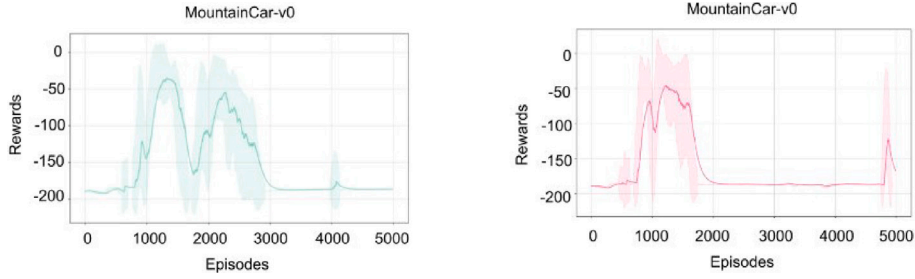


Fig. 7. Comparison of standard DQN and average priority thresholding DQN on the MountainCar-v0 environment. Left (in green): standard DQN. Right (in pink): DQN with average priority thresholding.

5.2. Average reward threshold evaluation

The MountainCar-v0 environment is evaluated for average reward thresholding DQN, consisting of two level buffers. A total of 10,000 samples is divided into 3000 and 7000 buffer levels. The 3000 buffer consists of high reward samples which are more than average and 7000 consists of less than average reward. With a batch size of 128 the model was able to show improvements in the range of 2000 to 4000 episodes. This is an improvement over classic DQN implementation as seen in Fig. 6, where the model exhibits a sharp performance collapse after 3000 episodes but average reward approach showed decline after 4000 episodes. This approach leads to high-reward state space distribution and enhances the overall learning of the policy. The approach focuses more on gaining momentum towards high-reward samples as naturally such distribution exploits better learning. Moreover, the unequal size of buffers where the high level buffer size is less than the low level buffer eventually removes some kind of bias which may arise by exploring just the high reward state distribution.

5.3. Average priority threshold evaluation

The MountainCar-v0 environment is evaluated for average priority thresholding DQN which consists of two buffer levels. A total of 10,000 samples is divided into 3000, being a high priority level and 7000, being a low priority level. The priority is the minimum Euclidean distance from all the samples in the buffer which is damped by a factor. The α values are tested for a range from 0.5 to 0.8 with a batch size of 128. Although no significant improvement was observed as seen in Fig. 7, the average priority thresholding DQN performed worse than classic DQN as the model tends to completely forget experience after 2000 as opposed to classic DQN which drops off after 3000. This technique is based on learning more from important experiences than from others. As a result, over-exploiting high-priority samples can cause the training to diverge from the true target distribution, leading the policy to forget relevant past experiences and overfit to a limited subset of the buffer.

5.4. Combined reward priority threshold evaluation

The MountainCar-v0 environment is evaluated by combining approaches from Sections 4.1 & 4.2. The priority is calculated as in the case of average priority thresholding and the samples are divided into levels 3000 & 7000 based on whether the sample reward is greater or less than the average buffer reward. The α values are tested for a range from 0.5 to 0.8 with a batch size of 128. Although no significant improvement was observed as seen in Fig. 8, the results were more similar to the average priority thresholding DQN; the model forgets experience after 2000 as opposed to classic DQN which drops off after 3000. The approach combines the momentum of high-reward samples and learning from important experiences. The high-reward and low-reward samples both tend to have high priorities because of high deviation in state space distribution. The prioritized sampling gives more importance to low level buffer and hence nullifies the effect of high reward thresholding.

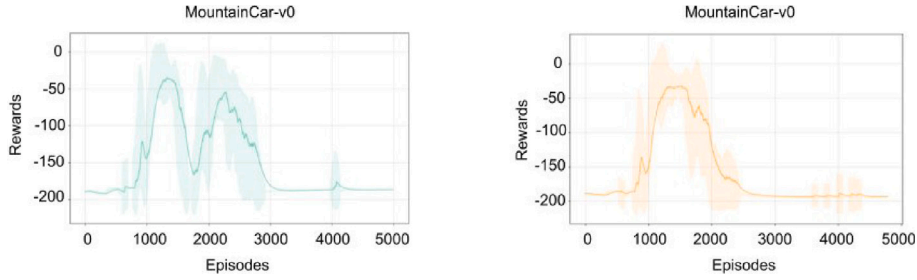


Fig. 8. Comparison of standard DQN and combined thresholding DQN on the MountainCar-v0 environment. Left (in green): standard DQN. Right (in orange): DQN with combined thresholding.

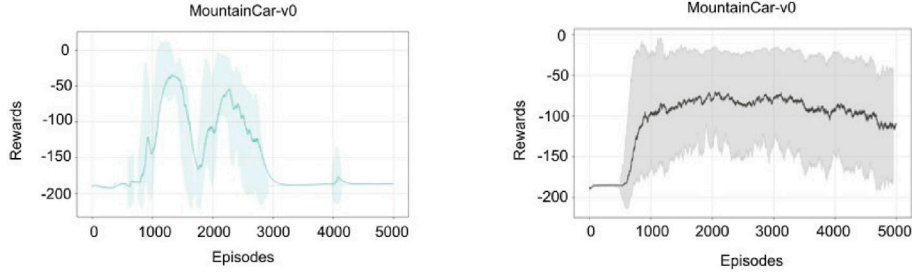


Fig. 9. Comparison of standard DQN and K-Means sampling DQN on the MountainCar-v0 environment. Left (in green): standard DQN. Right (in grey): K-Means sampling DQN.

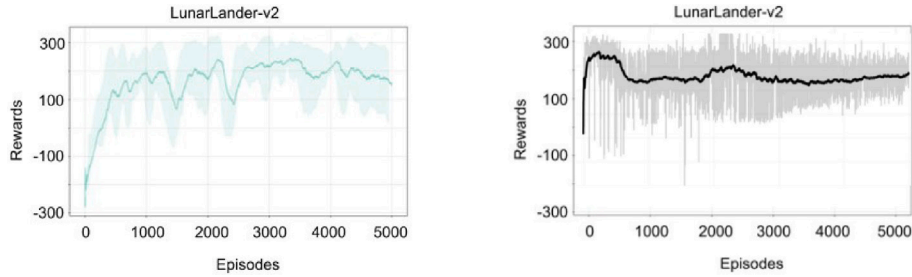


Fig. 10. Comparison of standard DQN and K-Means sampling DQN on the LunarLander-v2 environment. Left (in green): standard DQN. Right (in grey): K-Means sampling DQN.

5.5. K-means DQN evaluation

We test the MountainCar-v0 and LunarLander-v2 environments (the ones that exhibit the most noticeable catastrophic forgetting) using a Deep Q-Network (DQN) enhanced with K-Means Experience Replay (K-MER). The replay buffer size is set to 10,000, and the number of clusters is dynamically determined based on the batch size. This method completely mitigates catastrophic forgetting and consistently outperforms other experience replay strategies. As seen in Fig. 9, in the MountainCar-v0 environment, standard DQN shows clear signs of forgetting, with performance dipping twice; in contrast, K-MER remains stable from episode 1000 to around episode 5000, with only a slight dip near episode 4000 indicating negligible forgetting. K-MER clusters transitions in the state space, allowing the network to sample from semantically coherent regions. This effectively ensembles policy updates across diverse state clusters, promoting generalization and preserving critical transitions. As a result, the method maintains a robust balance between exploration and exploitation, with clustered samples refining the policy more effectively within their respective local regions.

Comparing K-MER with threshold-based strategies, the latter rely on buffer-wide averages that evolve during training. In the average-reward variant, the running reward threshold shifts as policy performance changes, repeatedly altering which transitions are replayed. In the priority-based variant, TD estimates are highly non-stationary, particularly in early training, so a global cutoff can magnify transient estimation noise. The combined strategy inherits both effects, producing a replay partition that changes as learning progresses. These moving boundaries can cause portions of the replay buffer to be sampled infrequently for extended periods, which reduces replay coverage and weakens value updates associated with those transitions.

Further, in case of LunarLander-v2 environment, catastrophic forgetting for the DQN based agent (as seen in green in Fig. 10) when trained using DQN can be attributed to the non-uniform distribution of experiences stored in the replay buffer. Although transitions are sampled uniformly at random, the buffer is populated according to the agent's state visitation patterns, which tend to concentrate on certain regions of the state space as learning progresses. Consequently, some frequently encountered flight states may exert a stronger influence on value updates, while less frequent but behaviorally important states may receive fewer updates over time. This imbalance can lead to gradual overwriting of earlier learned control behaviors, resulting in unstable value estimates and reduced robustness.

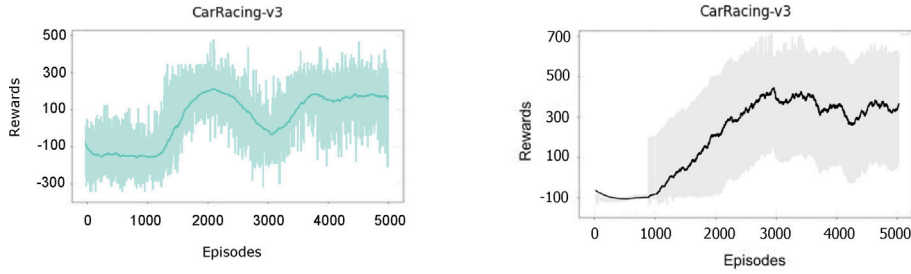


Fig. 11. Comparison of standard DQN and K-Means sampling DQN on the CarRacing-v3 environment. Left (in green): standard DQN. Right (in grey): K-Means sampling DQN.

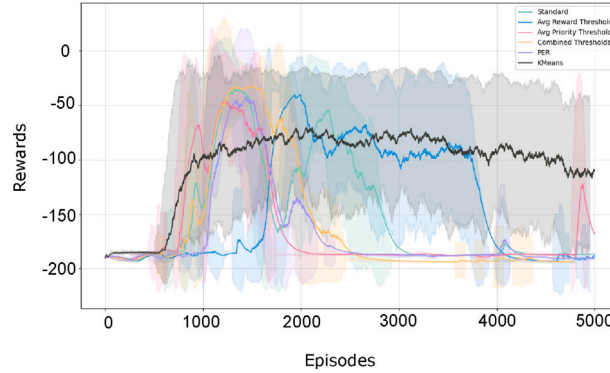


Fig. 12. Comparison of standard DQN with our proposed DQN variants and PER implementation. All the techniques are implemented in the MountainCar-v0 environment.

The proposed K-MER approach mitigates this issue by maintaining a balanced representation of experiences across different regions of the LunarLander state space (as seen in grey in Fig. 10), ensuring that both common and rare but important transitions continue to influence learning. As a result, K-MER preserves earlier learned control policies and maintains consistent performance over long training runs, reducing catastrophic forgetting in this environment.

Further, for higher-dimensional environments, we discuss observations of DQN vs K-MER on the CarRacing-v3 environment. It is a pixel-based continuous-control task in which the agent receives a raw RGB image of size $96 \times 96 \times 3$ at each timestep, corresponding to 27,648 input values per observation. The task is to control the vehicle so that it remains within track boundaries inferred purely from pixel observations. Unlike low-dimensional environments that provide structured state vectors, CarRacing-v3 requires the agent to learn control directly from images, including track layout and vehicle orientation. The large input size and strong temporal correlation between consecutive frames make learning difficult and increase the likelihood of forgetting previously learned behaviors during long training runs. Although DQN samples uniformly from the replay buffer, the buffer itself becomes dominated by visually similar recent trajectories, reducing coverage of rare but important driving states (such as curved sections of the track). This imbalance leads to unstable representation learning and policy drift. In contrast, the proposed K-MER approach limits interference between new and previously learned knowledge by preserving useful value information acquired earlier in training, preventing value drift during continued updates. This enables stable post-convergence learning and sustained high rewards without long-term performance collapse, even in a high-dimensional pixel-based environment (as seen in performance gap in Fig. 11).

5.6. Comparison with prevalent state-of-the-art

A detailed comparison of benchmark techniques is conducted among our various DQN variants. Fig. 12 shows comparison of standard DQN and Prioritized Experience Replay (PER) with different DQN variants, and as can be observed, the K-Means sampling strategy (K-MER) performs best overall, exhibiting minimal catastrophic forgetting compared to the other approaches. The PER implementation also performs similarly to priority threshold and combined reward priority threshold, showing complete forgetting after 2500 episodes.

Further, we compare our approach with clustering-based replay memory techniques to evaluate the impact of distribution-aware sampling on learning stability. The comparisons are drawn with prominent recent methods in the literature, including Clustering Experience Replay (CER) [19] and K-Means Replay Memory (KMRM) [20]. CER partitions the replay buffer into temporal segments and applies K-Means within each segment, while KMRM employs hierarchical buffers with loss-weighted cluster sampling. Although both methods improve short-term sampling diversity, their localized clustering often leads to uneven replay coverage and increased forgetting during longer training phases.

To ensure a fair and controlled comparison, all replay strategies (standard DQN, PER, CER, KMRM, and the proposed K-MER) operate within the same training pipeline. The neural network architecture, optimizer (Adam), learning rate, discount factor, replay

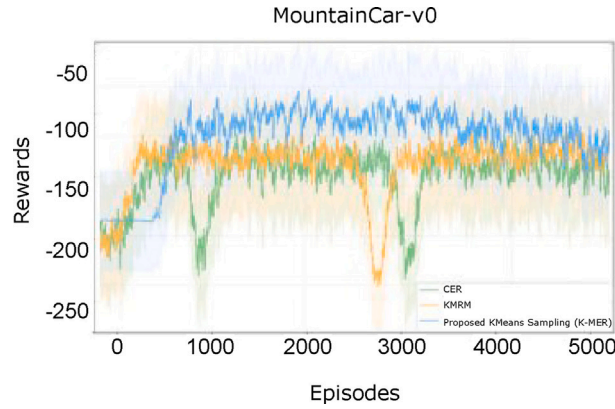


Fig. 13. Comparison of the proposed K-MER technique with clustering-based methods, CER and KMR. All techniques are implemented in the MountainCar-v0 environment.

Table 3

Quantitative comparison of K-MER with state-of-the-art techniques on MountainCar-v0 environment.

Method	Λ_p	Γ	$\mathbb{F}$
Standard DQN [1]	−190	≈ 700	2
PER [12]	−180	≈ 800	1
CER [19]	−150	≈ 1800	1
KMRM [20]	−140	≈ 2000	1
K-MER (ours)	−105	≈ 4800	NIL

buffer capacity (10,000), batch size, ϵ -greedy exploration schedule, and target network synchronization interval remain identical across all methods. Each model trains for the same number of episodes under identical environment seeds. The only difference between methods lies in how transitions are organized and sampled from the replay buffer.

For Prioritized Experience Replay (PER), transitions are sampled with probability proportional to their TD-error magnitude following Schaul et al. Sampling uses stochastic prioritization, and importance-sampling correction weights are applied during gradient updates to compensate for bias introduced by non-uniform sampling. For CER, the replay buffer is partitioned into fixed temporal segments that preserve insertion order. Within each segment, experiences are grouped according to similarity in state-action features, and representative samples are drawn from each group to construct a training batch. This segmentation limits grouping to localized time windows, consistent with the structure proposed in the original work. For KMRM, experiences are stored in a hierarchical replay structure composed of multiple sub-buffers. Each sub-buffer maintains a running average of the temporal-difference (TD) loss of its stored transitions. During batch construction, sub-buffers are selected with probability proportional to this average loss, and transitions are then sampled stochastically within the selected sub-buffer. This weighted hierarchical replay mechanism prioritizes regions of the buffer associated with higher learning signals while preserving stochastic sampling inside each sub-buffer.

As seen in Fig. 13, this limitation becomes evident as both CER and KMRM experience noticeable forgetting after approximately 2500–3000 episodes in the MountainCar-v0 environment, causing a decline in total reward and policy stability. In contrast, the proposed K-MER framework operates differently by clustering experiences across the entire state–action space rather than segmenting them by time or training error. This global clustering approach captures how experiences are distributed throughout all possible situations the agent may encounter, allowing more balanced sampling of both frequent and rare transitions. By maintaining this broad coverage, K-MER enables the agent to retain important but less common experiences, reducing forgetting and improving stability during extended training. Consequently, the method achieves faster convergence, more consistent learning behavior, and better long-term performance, demonstrating the benefits of distribution-aware replay in deep reinforcement learning.

Table 3 presents the quantitative comparison of the discussed techniques. Asymptotic performance (Λ_p) is computed as the average reward over the last 500 episodes, further averaged across 5 random seeds. Retention duration (Γ) denotes the average number of episodes after convergence during which performance remains above −110. $\mathbb{F}$ denotes the number of forgetting events, where a forgetting event is recorded whenever the running average reward—computed over a sliding window of 100 episodes—falls below −110 after the agent has previously exceeded this threshold.

6. Conclusion

In this work, we investigated state-action space aware sampling strategies to enhance experience replay and mitigate catastrophic forgetting in reinforcement learning. Although several environments from Classic Control, Box2D, and Atari were examined, significant catastrophic forgetting was observed only in the MountainCar-v0 environment. Therefore, all experimental evaluations were conducted on this environment to analyze learning stability and replay dynamics in depth. The proposed K-Means Experience Replay (K-MER) method was compared against Prioritized Experience Replay (PER) and recent clustering-based sampling techniques from

the literature, namely Clustering Experience Replay (CER) and K-Means Replay Memory (KMRM). The results demonstrate that K-MER provides more balanced state-action coverage and effectively reduces catastrophic forgetting by employing distribution-aware sampling.

For future work, we plan to extend our study to more diverse and challenging environments beyond Classic Control and Atari. This includes high-dimensional tasks and settings with partial observability, where stability and long-term memory are even more critical. Such exploration will provide a deeper understanding of the practical limitations and broader applicability of the proposed sampling strategies in real-world reinforcement learning problems.

CRedit authorship contribution statement

Armaan Garg: Writing – original draft, Validation, Supervision, Investigation, Conceptualization. **Ansh Raj Sharma:** Writing – original draft, Methodology, Formal analysis, Data curation. **Aryan Arora:** Validation, Investigation, Data curation. **Shilpa Verma:** Writing – review & editing, Supervision, Resources, Project administration.

Code availability

The code is publicly available and can be accessed from open repositories as cited in the manuscript.

Declaration of generative AI and AI-assisted technologies in the writing process

No AI-assisted writing or content generation tools were used in the preparation of this manuscript, and all written material is entirely the author's own work.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All datasets used in this study are publicly available and can be accessed from open repositories as cited in the manuscript.

References

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, et al., Human-level control through deep reinforcement learning, *Nature* 518 (2015) 529–533, <https://doi.org/10.1038/nature14236>
- [2] T.T. Nguyen, V.J. Reddi, Deep reinforcement learning for cyber security, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (2021) 3779–3795, <https://doi.org/10.1109/TNNLS.2021.3121870>
- [3] X. Wang, S. Wang, X. Liang, D. Zhao, J. Huang, X. Xu, B. Dai, Q. Miao, Deep reinforcement learning: a survey, *IEEE Trans. Neural Netw. Learn. Syst.* 35 (2022) 5064–5078, <https://doi.org/10.1109/TNNLS.2022.3207346>
- [4] L.-J. Lin, Self-improving reactive agents based on reinforcement learning, planning and teaching, *Mach. Learn.* 8 (1992) 293–321, <https://doi.org/10.1007/BF00992699>
- [5] D.E. Neves, L. Ishitani, Z.K.G. do Patrocínio Júnior, Advances and challenges in learning from experience replay, *Artif. Intell. Rev.* 58 (2024) 54, <https://doi.org/10.1007/s10462-024-11062-0>
- [6] A. Skrynnik, A. Staroverov, E. Aitygulov, K. Aksenov, V. Davydov, A.I. Panov, Forgetful experience replay in hierarchical reinforcement learning from expert demonstrations, *Knowl.-based Syst.* 218 (2021) 106844, <https://doi.org/10.1016/j.knosys.2021.106844>
- [7] X. Gong, J. Yu, S. Lü, H. Lu, Actor-critic with familiarity-based trajectory experience replay, *Inf. Sci.* 582 (2021) 633–647, <https://doi.org/10.1016/j.ins.2021.10.031>
- [8] C.J.C.H. Watkins, P. Dayan, Q-learning, *Mach. Learn.* 8 (1992) 279–292, <https://doi.org/10.1007/bf00992698>
- [9] OpenAI, Gymnasium API documentation, 2024, <https://gymnasium.farama.org/api/env/> (Accessed: 2 July 2025).
- [10] M. McCloskey, N.J. Cohen, Catastrophic interference in connectionist networks: the sequential learning problem, *Psychol. learn. motiv.* 24 (1989) 109–165, [https://doi.org/10.1016/s0079-7421\(08\)60536-8](https://doi.org/10.1016/s0079-7421(08)60536-8)
- [11] T. Zhang, X. Wang, B. Liang, B. Yuan, Catastrophic interference in reinforcement learning: a solution based on context division and knowledge distillation, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (2022) 9925–9939, <https://doi.org/10.1109/TNNLS.2022.3162241>
- [12] T. Schaul, J. Quan, I. Antonoglou, D. Silver, Prioritized experience replay, 2016. [arXiv:1511.05952](https://arxiv.org/abs/1511.05952).
- [13] R. Liu, J. Zou, The effects of memory replay in reinforcement learning, in: 2018 56th Annual Allerton Conference on Communication, Control, and Computing (Allerton), 2018, pp. 478–485, <https://doi.org/10.1109/ALLERTON.2018.8636075>
- [14] H. Hassani, S. Nikan, A. Shami, Improved exploration–exploitation trade-off through adaptive prioritized experience replay, *Neurocomputing* 614 (2025), <https://doi.org/10.1016/j.neucom.2024.128836>
- [15] H. Yamani, Y. Xing, L.V.C. Ong, B.A. MacDonald, H. Williams, Reward prediction error prioritisation in experience replay: the rpe-per method, 2025. [arXiv:2501.18093](https://arxiv.org/abs/2501.18093).
- [16] S.K. Sujit, S. Nath, P. Braga, S.E. Kahou, Prioritizing samples in reinforcement learning with reducible loss, in: Thirty-Seventh Conference on Neural Information Processing Systems, 2023, <https://openreview.net/forum?id=g78QqvhvDU>.
- [17] J.I. Kim, S.B. Kim, D-qmix: multi-step sequential forward dynamics modeling with global state and self-attention for sample-efficient multi-agent reinforcement learning, *Inf. Sci.* 729 (2025) 122867, <https://doi.org/10.1016/j.ins.2025.122867>
- [18] Y. Gu, Z. Liang, S. Wu, Y. Tao, C. Zhao, R. Xu, Sample-efficient multi-objective reinforcement learning for hybrid impulsive information consensus in multi-agent systems, *Inf. Sci.* 727 (2025) 122791, <https://doi.org/10.1016/j.ins.2025.122791>
- [19] M. Li, T. Huang, W. Zhu, Clustering experience replay for the effective exploitation in reinforcement learning, *Pattern Recognit.* 131 (2022) 108875, <https://doi.org/10.1016/j.patcog.2022.108875>
- [20] S. Serbest, A.S. Taşbaş, S.O. Şahin, Improving experience replay architecture with K-means clustering, in: 2023 31st Signal Processing and Communications Applications Conference (SIU), 2023, pp. 1–4, <https://doi.org/10.1109/SIU59756.2023.10224023>
- [21] M.R.R. Tushar, S. Siddique, A memory efficient deep reinforcement learning approach for snake game autonomous agents, in: 2022 IEEE 16th International Conference on Application of Information and Communication Technologies (AICT), 2022, pp. 1–6, <https://doi.org/10.1109/AICT55583.2022.10013603>